Inter-Process Communication (IPC) with Shared Memory and Message Passing

Objectives

- Understand the concept of **Inter-Process Communication (IPC)**.
- Differentiate between **Shared Memory** and **Message Passing**.
- Implement IPC using **system calls in C**.
- Write simple producer–consumer style programs using IPC mechanisms.

Inter-Process Communication (IPC)

IPC allows processes to exchange data and synchronize execution. Two major IPC models are:

1. **Message Passing:** Processes communicate by sending and receiving messages via the kernel.
2. **Shared Memory:** Processes share a portion of memory that they can both read/write.

Shared Memory IPC:

Shared memory is a highly efficient IPC method where multiple processes can access a common memory segment. This allows communication to be as fast as memory access, as it avoids the need for data to be copied through the kernel. However, this method requires explicit synchronization by the programmer.

Methods and Commands

The following are key System V IPC functions for managing shared memory:
Key system calls:

- `shmget()` → Create a shared memory segment
- `shmat()` → Attach shared memory to process
- `shmdt()` → Detach shared memory
- `shmctl()` → Control (remove) shared memory

- `ftok()` → Generates a unique key for a shared resource.

Quick prerequisites (one-time)

1. Install build tools (if not already):

```
sudo apt update
sudo apt install build-essential
```

2. Create small files used by `ftok()` (if code uses them):

```
touch progfile shmfile
```

(`ftok()` needs an existing pathname.)

3. Helpful utilities:

`ipcs` — list IPC resources

`ipcrm` — remove IPC resources

```
ipcs -q    # message queues
ipcs -m    # shared memory
ipcs -s    # semaphores
```

PART A — Shared Memory Example

Safe writer / reader example
This program demonstrates the basic flow of a writer process placing data into a shared memory segment and a reader process retrieving it.

C Program Code:

shm_writer.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    key_t key = ftok("shmfile", 65);          // ensure "shmfile" exists
    if (key == -1) { perror("ftok"); exit(1); }

    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    if (shmid == -1) { perror("shmget"); exit(1); }

    char *data = (char *) shmat(shmid, NULL, 0);
    if (data == (char *)(-1)) { perror("shmat"); exit(1); }

    printf("Enter text to write to shared memory (max 1023 chars):\n> ");
    if (!fgets(data, 1024, stdin)) data[0] = '\0';

    // remove trailing newline if present
    size_t n = strlen(data);
    if (n && data[n-1] == '\n') data[n-1] = '\0';

    printf("Data written: '%s'\n", data);

    if (shmdt(data) == -1) perror("shmdt");
    return 0;
}
```

Shm_reader.c

shm_reader.c

```c
// shm_reader.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    key_t key = ftok("shmfile", 65);
    if (key == -1) { perror("ftok"); exit(1); }

    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    if (shmid == -1) { perror("shmget"); exit(1); }

    char *data = (char *) shmat(shmid, NULL, 0);
    if (data == (char *)(-1)) { perror("shmat"); exit(1); }

    printf("Data read from shared memory:\n> %s\n", data);

    if (shmdt(data) == -1) perror("shmdt");
    // remove segment after reading (optional / instructor decision)
    if (shmctl(shmid, IPC_RMID, NULL) == -1) perror("shmctl(IPC_RMID)");

    return 0;
}
```

Compile & Run (step-by-step)

1. **Compile:**

```
gcc shm_writer.c -o shm_writer
gcc shm_reader.c -o shm_reader
```

2. **Ensure `shmfile` exists:**

```
touch shmfile
```

3. **Run writer in Terminal A:**

```
./shm_writer
```

4. **In Terminal B run reader:**

```
./shm_reader
```

   Reader prints the text. If the reader runs before the writer, it may see empty content or block until something writes.

Clean up (if leftover):

1. List:

```
ipcs -m
```

2. Remove by `shmid` (if needed):

```
ipcrm -m <shmid>
```

PART B — Synchronization for Shared Memory (Semaphores)

A **semaphore** is a synchronization tool that controls access to shared resources. A **binary semaphore** acts like a lock, enforcing mutual exclusion to ensure only one process can enter a critical section at a time. A **counting semaphore** manages a pool of resources, allowing a specified number of processes to access them concurrently. The core operations are

wait() (or P), which decrements the semaphore's value and blocks if it becomes negative, and signal() (or V), which increments the value and unblocks a waiting process.

Key System Calls for Semaphores:

`semget()` →Creates a new semaphore set or gets the ID of an existing one.

`semop()` →Performs an operation on a semaphore, such as `wait` (sem_op < 0) or `signal` (sem_op > 0).

`semctl()` →Performs control operations like initializing a semaphore's value (`SETVAL`) or removing the set (`IPC_RMID`).

LAB TASK:The Producer-Consumer Problem

Objective: Implement a classic synchronization problem using a shared memory buffer and a semaphore to ensure mutual exclusion, thereby preventing race conditions.

Problem Statement: Write a C program that computes the sum of the first 1,000,000 positive integers.

1. The parent process will create a shared memory segment for a single integer variable to hold the total sum.
2. The parent will also create and initialize a binary semaphore to ensure mutual exclusion.
3. The parent will then fork ten child processes.
4. Each child process will be responsible for computing the sum of a range of integers (e.g., the first child sums 1 to 100,000, the second sums 100,001 to 200,000, etc.).
5. To update the shared total sum, each child process must first acquire the semaphore using a wait operation, add its partial sum to the shared total, and then release the semaphore with a signal operation.
6. The parent process will wait for all child processes to terminate and then print the final sum.
7. The parent must also ensure that the shared memory segment and the semaphore set are correctly removed from the system after all processes have finished.

PART C —Message Passing IPC

Message passing is an IPC model where processes communicate by exchanging messages through a kernel-managed message queue. A message queue is a linked list of messages , and communication is decoupled, meaning the sender and receiver do not need to be running at the same time. Messages can also be retrieved selectively based on their type.

Methods and Commands
The following are key System V IPC functions for managing message queues:

`ftok()`→Generates a unique key that all processes use to identify the message queue.
`msgget()`→Creates or gets a message queue identifier.
`msgsnd()`→Sends a message to a message queue.
`msgrcv()`→Receives a message from a message queue.
`msgctl()`→Performs control operations on the message queue, such as deletion

Example Task: Message Queue Sender & Receiver

This program demonstrates simple, decoupled communication between a sender and a receiver process using a message queue.

C Program Code:

msg_sender.c

C msg_sender.c

```c
// msg_sender.c
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAXMSG 128

struct msgbuf {
    long mtype;
    char mtext[MAXMSG];
};

int main() {
    key_t key = ftok("progfile", 65); // ensure progfile exists
    if (key == -1) { perror("ftok"); exit(1); }

    int msqid = msgget(key, IPC_CREAT | 0666);
    if (msqid == -1) { perror("msgget"); exit(1); }

    struct msgbuf sbuf;
    sbuf.mtype = 1;

    printf("Enter message (max %d chars):\n> ", MAXMSG-1);
    if (!fgets(sbuf.mtext, MAXMSG, stdin)) sbuf.mtext[0] = '\0';
    size_t len = strlen(sbuf.mtext);
    if (len && sbuf.mtext[len-1] == '\n') sbuf.mtext[len-1] = '\0';

    if (msgsnd(msqid, &sbuf, strlen(sbuf.mtext)+1, 0) == -1) {
        perror("msgsnd"); exit(1);
    } else {
        printf("Message sent.\n");
    }
    return 0;
}
```

msg_receiver.c

msg_receiver.c

```c
// msg_receiver.c
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <stdlib.h>
#define MAXMSG 128

struct msgbuf {
    long mtype;
    char mtext[MAXMSG];
};

int main() {
    key_t key = ftok("progfile", 65);
    if (key == -1) { perror("ftok"); exit(1); }

    int msqid = msgget(key, 0666);
    if (msqid == -1) { perror("msgget"); exit(1); }

    struct msgbuf rbuf;
    if (msgrcv(msqid, &rbuf, MAXMSG, 1, 0) == -1) {
        perror("msgrcv"); exit(1);
    }

    printf("Received: %s\n", rbuf.mtext);

    // cleanup
    if (msgctl(msqid, IPC_RMID, NULL) == -1) perror("msgctl IPC_RMID");
    return 0;
}
```

Compile & run

1. **Compile:**

```
gcc msg_sender.c -o msg_sender
gcc msg_receiver.c -o msg_receiver
```

2. **Ensure `progfile` exists:**

```
touch progfile
```

3. **Start receiver in Terminal B:**

```
./msg_receiver
```

It blocks waiting for message type 1.

4. **In Terminal A send a message:**

```
./msg_sender
```

Receiver prints the message and removes the queue.
Note: If sender runs first, message stays in queue until the receiver runs.

LAB TASK: Message Passing Task

Objective: To implement a basic task dispatch system using a message queue, demonstrating the use of message types.

Problem Statement: Write a C program that simulates a simple command-line task dispatcher.

1. Create a "server" process that creates and manages a single message queue.
2. Create a "client" process that sends messages to the queue.
3. The client process will send messages with different message types (mtype) to represent different commands. For example:
    - mtype = 1: "PRINT_INFO" (e.g., print system uptime).
    - mtype = 2: "EXEC_COMMAND" (the mtext field will contain a command string to be executed).
    - mtype = 3: "EXIT" (a command to shut down the server).
4. The server process will enter a continuous loop, receiving messages from the queue using msgrcv().
5. Based on the mtype of the received message, the server will perform the corresponding action. For example, for type 2, it will use system() to execute the command in the message.
6. When the server receives an "EXIT" message, it should clean up the message queue and terminate gracefully.

Submission:

Submit one ZIP named `<RollNo>_LAB_05`, all `.c` source files, `README` (exact compile/run commands), and screenshots of compile/run and `ipcs` showing cleanup.